

LCI · Le Comptoir Immo

DE · SUPPORT / EXPLOITATION

Dossier d'Exploitation

Surveillance · mises à jour · sauvegardes ·
dépannage

FastAPI · React/Vite | Docker Compose | PostgreSQL · Nginx · Let's
Encrypt

Projet : Le Comptoir Immo & Alice · **Version :** 1.0 · **Date :** 3 juin 2026

Domaines : immo.lecomptoir.services · alice.lecomptoir.services

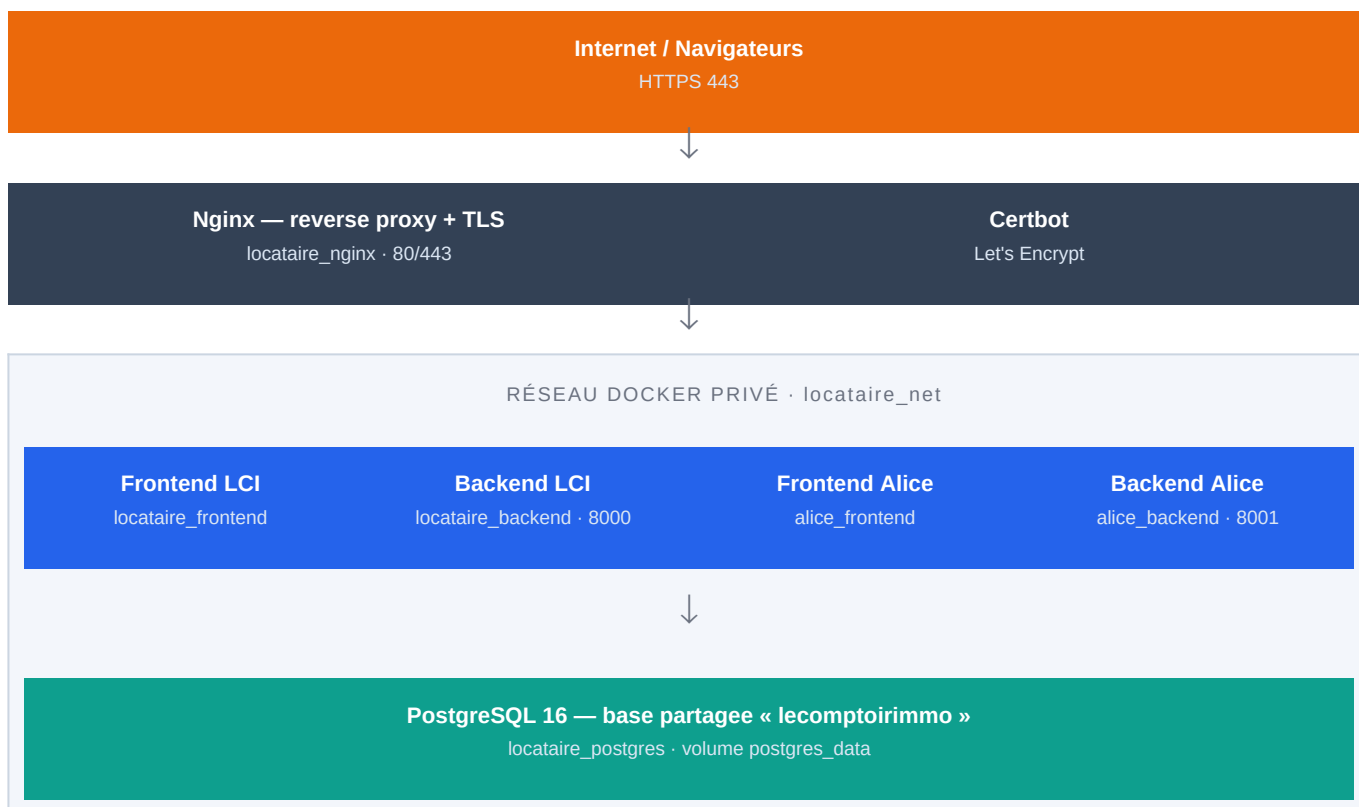
Document confidentiel. Aucun secret inclus — les valeurs sensibles
vivent dans les .env.prod (hors Git).

1 Accès & repères

Exploitation courante : surveillance, mises à jour, sauvegardes, restauration, rollback et dépannage.

Élément	Valeur
Serveur	OVH AlmaLinux · IP 91.134.138.246
Accès SSH	ssh lecomptoir-vps · port 58007 · user almalinux · sudo nopasswd
Dossier projet	~/LeComptoirImmo
Stack	docker/docker-compose.prod.yml
Domaines	immo.lecomptoir.services · alice.lecomptoir.services
Dépôt	git@github.com:service-lecomptoir/LeComptoirImmo.git (main)

Raccourci sur le serveur : **alias dc='docker compose -f docker/docker-compose.prod.yml'**



Volumes persistants : **postgres_data** (base) · **uploads_data** (pièces jointes) · **certbot_certs** / **certbot_www** (TLS). Seul Nginx expose des ports publics ; Postgres n'est jamais exposé.

2 Surveillance & santé

```
dc ps # état (Up / healthy)
dc logs --tail=100 backend # logs applicatifs
dc stats --no-stream # CPU / RAM
df -h ; docker system df # espace disque
```

Lecture des codes

Un **403** sur `/api/v1/settings/scheduler` est **normal** (endpoint protégé) → backend OK. Un **502/504** côté Nginx = backend down ou en cours de démarrage.

3 Mise à jour de l'application

Méthode standard (depuis le dépôt)

```
cd ~/LeComptoirImmo
git fetch origin && git checkout main && git pull --ff-only
dc up -d --build backend frontend # + alice-backend alice-frontend si modifiés
dc ps
```

Règles de déploiement

- Ne préciser que les **services modifiés** (évite de recréer Postgres).
- **Éviter** `--force-recreate` : recréer `backend` recrée `postgres` → coupure DB.
- Après tout changement de `.env.prod` : `up -d --build` (un `restart` ne recharge pas les `env_file`).

Déploiement « à chaud » depuis Windows (sans Git serveur)

```
$tgz = "env:TEMP\leci_deploy.tgz"; if (Test-Path $tgz) { Remove-Item $tgz }
tar --format=ustar -czf $tgz backend/app frontend/src
scp $tgz lecomptoir-vps:~/leci_deploy.tgz
ssh lecomptoir-vps "sudo tar --overwrite -xzf ~/leci_deploy.tgz -C ~/LeComptoirImmo && sudo chown -R
almalinux:almalinux ~/LeComptoirImmo/backend/app ~/LeComptoirImmo/frontend/src"
ssh lecomptoir-vps "cd ~/LeComptoirImmo && docker compose -f docker/docker-compose.prod.yml up -d --build
backend frontend"
```

Format **ustar** (en-têtes compatibles) ; extraction **sudo** + **chown** pour les dossiers en lecture seule. Pas de cmdlets PowerShell dans la commande **ssh**.

4 Rollback / bascule (retour arrière)

Avant tout rollback

Faire une **sauvegarde DB** (§5). Un retour de code ne défait pas une migration de données déjà appliquée.

Rollback du code

```
cd ~/LeComptoirImmo
git fetch origin && git log --oneline -10 # repérer le dernier commit stable <SHA>
git checkout <SHA> # (après sauvegarde)
dc up -d --build backend frontend alice-backend alice-frontend
```

Puis revenir à la branche : `git checkout main && git pull --ff-only`.

Rollback d'image (sans rebuild)

```
docker images | grep docker-backend # repérer un Image ID antérieur
docker tag <IMAGE_ID> docker-backend:latest
dc up -d backend
```

Restauration base (rollback données)

```
dc stop backend alice-backend
cat ~/backups/leci_<date>.sql | dc exec -T postgres sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"'
dc start backend alice-backend
```

5 Sauvegardes

```
mkdir -p ~/backups
# Base
dc exec -T postgres sh -c 'pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" > ~/backups/leci_$(date +%F).sql'
# Uploads
docker run --rm -v lecomptoirimmo_uploads_data:/data -v ~/backups:/backup alpine \
tar czf /backup/uploads_$(date +%F).tgz -C /data .
```

Bonnes pratiques

Copier `~/backups/` hors-serveur régulièrement. **Tester une restauration** périodiquement. Vérifier le préfixe de volume avec `docker volume ls`.

6 Automatisation (cron)

Éditer la crontab : `crontab -e`

```
# Sauvegarde DB quotidienne – 02h30
30 2 * * * cd ~/LeComptoirImmo && docker compose -f docker/docker-compose.prod.yml exec -T postgres sh -c
'pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" > ~/backups/leci_$(date +%F).sql 2>> ~/backups/backup.log'
# Sauvegarde uploads – 02h45
45 2 * * * docker run --rm -v lecomptoirimmo_uploads_data:/data -v ~/backups:/backup alpine tar czf
/backup/uploads_$(date +%F).tgz -C /data . 2>> ~/backups/backup.log
# Purge > 30 jours – 03h15
15 3 * * * find ~/backups -name 'leci_*.sql' -mtime +30 -delete ; find ~/backups -name 'uploads_*.tgz'
-mtime +30 -delete
# Renouvellement TLS – 1er du mois 04h00
0 4 1 * * cd ~/LeComptoirImmo && docker compose -f docker/docker-compose.prod.yml run --rm certbot renew
&& docker compose -f docker/docker-compose.prod.yml exec nginx nginx -s reload
```

Échappement cron

Dans une crontab, % doit être échappé en `\%` (cf. `date +%F` ci-dessus).

7 Conteneurs & TLS

Commande	Effet
<code>dc restart backend</code>	Redémarrage simple (ne recharge pas les env_file)
<code>dc up -d --build backend</code>	Rebuild + recréation (recharge .env.prod)
<code>dc stop</code> / <code>dc start</code>	Arrêt / démarrage de la stack
<code>dc down</code>	Supprime les conteneurs — volumes CONSERVÉS
<code>dc run --rm certbot renew</code>	Renouvelle les certificats TLS

Ne jamais

Utiliser `dc down -v` en production : cela **efface la base et les uploads**. Ni `docker volume prune` à l'aveugle.

8 Rôles & isolation

Rôles : `admin` `gestionnaire` `GP` `propriétaire` `locataire`.

Isolation appliquée (listes ET accès par identifiant)

Un **GP** ne voit que ses ressources ; un **mandataire** ne voit pas celles d'un GP ; **propriétaire/locataire** limités à leur périmètre. Vérifié sur paiements, avis, documents, biens, locataires et fiches propriétaires (lecture + écriture + téléchargements).

9 E-mails (SMTP)

L'envoi est **désactivé tant que SMTP_HOST est vide** : les documents restent consultables dans l'app, aucun e-mail ne part. Pour activer : renseigner `SMTP_*` dans `backend/.env.prod` puis `dc up -d --build backend`.

10 Dépannage rapide (FAQ)

Symptôme	Cause probable	Action
502/504	Backend down / démarrage / crash	<code>dc ps ; dc logs --tail=100 backend</code>
Backend ne démarre pas	Clé inconnue dans <code>.env.prod</code>	Corriger ; <code>dc up -d --build backend</code>
Alice <code>unhealthy</code>	Postgres recréé (dépendance)	<code>dc up -d alice-backend</code>
Liaison LCI ↔ Alice KO	Clés internes différentes	Aligner ; rebuild des 2 backends
Modif <code>.env.prod</code> sans effet	<code>restart</code> ne recharge pas	<code>dc up -d --build <service></code>
Disque plein	Images orphelines	<code>docker image prune -f</code>
Certificat expiré	Renouvellement non fait	<code>certbot renew</code> + reload Nginx
Quittance non reçue	SMTP désactivé	Activer SMTP (§9)

Redémarrage propre (incident généralisé)

```
cd ~/LeComptoirImmo
dc logs --tail=200 backend alice-backend
dc up -d # recrée les conteneurs manquants (volumes intacts)
dc up -d --build backend frontend alice-backend alice-frontend # si besoin
```

11 Bonnes pratiques

- Sauvegarde DB quotidienne + **test de restauration** périodique.
- Déployer **service par service**, vérifier la santé après chaque déploiement.
- Ne jamais committer de secret ; `*.env.prod` dans le coffre de l'équipe.
- Garder Postgres hors d'accès public.
- Documenter chaque incident et sa résolution.